

Мета роботи:

Засвоєння понять ентропії на символ джерела та його надлишковості, вивчення та порівняння різних моделей джерела відкритого тексту для наближеного визначення ентропії, набуття практичних навичок щодо оцінки ентропії на символ джерела.

Хід роботи:

1. Підготовка тексту

Виконайте попередню обробку тексту відповідно до таких правил:

- усі літери привести до нижнього регістру;
- символи, що не належать до заданого алфавіту, вилучити або замінити пробілами;
- послідовності пробілів та інших розділових символів трактувати як один пробіл;
- для обчислень без пробілів вилучити всі пробіли з попередньо нормалізованого тексту.

Зафіксуйте правила попередньої обробки, використані у вашій програмі.

```
part1.py > ...
1  import re
2
3  input = "text.txt"
4
5  normalized = "normalized.txt"
6  no_spaces = "withoutSpaces.txt"
7
8  alphabet = "абвгдеёжзийклмнопрстуфхцчшщъьзюя"
9
10
11 def normalize_text(text):
12     text = text.lower()
13     text = re.sub(f"[^{alphabet}\\s]", " ", text)
14     text = re.sub(r"\s+", " ", text)
15     text = text.strip()
16     return text
17
18
19 with open(input, "r", encoding="utf-8") as file:
20     text = file.read()
21
22 normalized_text = normalize_text(text)
23 text_without_spaces = normalized_text.replace(" ", "")
24
25 with open(normalized, "w", encoding="utf-8") as file:
26     file.write(normalized_text)
27
28 with open(no_spaces, "w", encoding="utf-8") as file:
29     file.write(text_without_spaces)
30
```

У програмі передбачено такі правила попередньої обробки тексту:

1) У весь текст переводиться у нижній регістр, щоб однакові літери у верхньому та нижньому регістрах вважалися одним символом.

2) Для аналізу використовується алфавіт, що складається з 33 літер російської мови:

абвгдеёжзийклмнопрстуфхцчшщъыьэюя.

3) Символи, які не належать до заданого алфавіту та не є пробільними символами, замінюються на пробіл. Таким чином, цифри, розділові знаки, латинські літери та інші сторонні символи не беруть участі в подальшому аналізі.

4) Послідовності пробілів та інших пробільних символів (зокрема перенесень рядків і табуляцій) замінюються одним пробілом.

5) Видаляються пробіли на початку та в кінці тексту.

6) Після нормалізації створюються два варіанти тексту:

- нормалізований текст із пробілами — зберігається у файлі `normalized.txt`;

- нормалізований текст без пробілів — усі пробіли видаляються, а результат зберігається у файлі `withoutSpaces.txt`.

2. Розрахунок статистичних характеристик тексту

Реалізуйте програму для:

1. підрахунку частот окремих символів

Реалізується формула з методички ($H = -\sum p_i \log_2(p_i)$)

Ця формула працює так що для початку треба знайти ймовірність (а саме частоту розділити на довжину тексту) й потім помножити на $\log_2$ та сумувати зі знаком(-)

Для зручності написали 2 окремих скрипта:

підрахунок частот окремих символів;

обчислення H_1 за частотами окремих символів;

```
import math
from collections import Counter

normalized = "normalized.txt"
no_spaces = "withoutSpaces.txt"

def symbol_frequencies(text):
    return Counter(text)

def calculate_h1(text):
    frequencies = symbol_frequencies(text)
    total = len(text)
    h1 = 0

    for count in frequencies.values():
        p = count / total
        h1 -= p * math.log2(p)

    return h1

def print_statistics(text, name):
    print(name)

    frequencies = symbol_frequencies(text)
    total = len(text)

    print("\nТоп-5 символів:")
```

```
    for symbol, count in frequencies.most_common(5):
        if symbol == " ":
            symbol = "space"

        probability = count / total

        print(f"{symbol}: {count}, p = {probability:.6f}")

    h1 = calculate_h1(text)

    print(f"\nH1 = {h1:.6f} bit")

with open(normalized, "r", encoding="utf-8") as file:
    normalized_text = file.read()

with open(no_spaces, "r", encoding="utf-8") as file:
    text_without_spaces = file.read()

print_statistics(
    normalized_text,
    "\nNormalized"
)

print_statistics(
    text_without_spaces,
    "\nWithoutSpaces"
)
```

Результат:

```

PS D:\Projects\crypto> & C:\msys64\ucrt64\bin\python.exe d:/Projects/crypto/part2-1.py

Normalized

Топ-5 символів:
space: 152025, p = 0.162167
o: 85259, p = 0.090947
a: 65264, p = 0.069618
e: 60686, p = 0.064734
и: 55156, p = 0.058836

H1 = 4.375971 bit

WithoutSpaces

Топ-5 символів:
o: 85259, p = 0.108550
a: 65264, p = 0.083093
e: 60686, p = 0.077264
и: 55156, p = 0.070224
н: 49254, p = 0.062709

H1 = 4.459721 bit
PS D:\Projects\crypto>

```

2 Скрипт:

підрахунку частот біграм;

обчислення H_2 за частотами біграм.

```

1  import math
2  from collections import Counter
3
4
5  normalized = "normalized.txt"
6  no_spaces = "withoutSpaces.txt"
7
8
9  def bigram_frequencies(text, step):
10
11      bigrams = []
12
13      for i in range(0, len(text) - 1, step):
14
15          bigrams.append(text[i:i + 2])
16
17      return Counter(bigrams)
18
19
20  def calculate_h2(text, step):
21
22      frequencies = bigram_frequencies(text, step)
23
24      total = sum(frequencies.values())
25
26      h2 = 0
27
28      for count in frequencies.values():
29
30          p = count / total
31
32          h2 -= p * math.log2(p)
33
34      return h2 / 2

```

```

37 def print_statistics(text, name):
38
39     print(name)
40
41     # Біграми, що перетинаються
42
43     overlap = bigram_frequencies(text, 1)
44     print("\nТоп-5 біграм, що перетинаються:")
45     total = sum(overlap.values())
46
47     for bigram, count in overlap.most_common(5):
48         probability = count / total
49         display_bigram = bigram.replace(" ", "[space]")
50         print(f"{display_bigram}: {count}, p = {probability:.6f}")
51
52     h2_overlap = calculate_h2(text, 1)
53
54     print(
55         f"\nH2 = "
56         f"{h2_overlap:.6f} біт"
57     )
58
59     # Біграми, що не перетинаються
60
61     non_overlap = bigram_frequencies(text, 2)
62
63     print("\nТоп-5 біграм, що не перетинаються:")
64
65     total = sum(non_overlap.values())

```

```

67     for bigram, count in non_overlap.most_common(5):
68
69         probability = count / total
70
71         display_bigram = bigram.replace(" ", "[space]")
72
73         print(f"{display_bigram}: {count}, p = {probability:.6f}")
74
75     h2_non_overlap = calculate_h2(text, 2)
76
77     print(
78         f"\nH2 = "
79         f"{h2_non_overlap:.6f} біт"
80     )
81
82
83     with open(normalized, "r", encoding="utf-8") as file:
84         normalized_text = file.read()
85
86
87     with open(no_spaces, "r", encoding="utf-8") as file:
88         text_without_spaces = file.read()
89
90
91
92
93     print_statistics(
94         normalized_text,
95         "\nNormalized"
96     )

```

Результат:

```
PS D:\Projects\crypto> & C:\msys64\ucrt64\bin\python.exe d:/Projects/crypto/part2-2.py

Normalized

Топ-5 біграм, що перетинаються:
и[space]: 18829, p = 0.020085
о[space]: 18228, p = 0.019444
а[space]: 16668, p = 0.017780
[space]н: 16127, p = 0.017203
е[space]: 14798, p = 0.015785

H2 = 3.995763 bit

Топ-5 біграм, що не перетинаються:
и[space]: 9460, p = 0.020182
о[space]: 9102, p = 0.019418
а[space]: 8346, p = 0.017806
[space]н: 8023, p = 0.017116
е[space]: 7397, p = 0.015781

H2 = 3.995578 bit

WithoutSpaces

Топ-5 біграм, що перетинаються:
то: 10632, p = 0.013536
на: 9327, p = 0.011875
он: 9234, p = 0.011757
ст: 9076, p = 0.011555
по: 8765, p = 0.011159

H2 = 4.167906 bit

Топ-5 біграм, що не перетинаються:
то: 5394, p = 0.013735
он: 4693, p = 0.011950
на: 4631, p = 0.011792
ст: 4518, p = 0.011504
по: 4434, p = 0.011291

H2 = 4.167725 bit
PS D:\Projects\crypto>
```

Текст	Перекривні(H2)	Неперекривні(H2)
Є пробіли	3.995763	3.995578
Без пробілів	4.167906	4.167725

Значення H_2 для перекривних і неперекривних біграм майже не відрізняються.

Це пояснюється великим обсягом тексту, через що статистичні розподіли в обох випадках виходять дуже близькими. Перекривний спосіб використовує більше статистичних даних, оскільки бере біграми зі зміщенням на один символ, тому їх приблизно вдвічі більше, ніж неперекривних.

3. Дослідження впливу розподілу символів на ентропію

Наш прогноз:

А-фрагмент природного російськомовного тексту;

Невизначеність тут буде не максимальна, але більша за 0

В — послідовність, утворену повторенням одного символу;

Тут невизначеність взагалі відсутня, оскільки йде повтор одного й того самого символу

В — випадкову послідовність символів того самого алфавіту, в якій усі символи генеруються з однаковою ймовірністю.

Ось тут має бути максимальна невизначеність (тобто $\max H_1$)

Тепер перевіримо експериментально:

Для випадку А будемо використовувати звичайний нормалізований текст

Створюємо два файли (В та С):

В буде містити такий текст:

aa

С буде містити такий текст:

абвгдеёжзийклмнопрстуфхцчщъыьэюя

Тепер до написання коду:

```
import math
from collections import Counter

normalized = "normalized.txt"
b_file = "B.txt"
c_file = "C.txt"

def symbol_frequencies(text):
    return Counter(text)

def calculate_h1(text):
    frequencies = symbol_frequencies(text)
    total = len(text)
    h1 = 0
    for count in frequencies.values():
        p = count / total
        h1 -= p * math.log2(p)

    return h1

def print_statistics(text, name):
    print(name)
    h1 = calculate_h1(text)
    print(f"H1 = {h1:.6f} bit")

with open(normalized, "r", encoding="utf-8") as file:
    natural_text = file.read()

with open(b_file, "r", encoding="utf-8") as file:
    b_text = file.read()
```

```

with open(c_file, "r", encoding="utf-8") as file:
    c_text = file.read()

print_statistics(
    natural_text,
    "\nПриродний текст"
)

print_statistics(
    b_text,
    "\nПовторення одного символу"
)

print_statistics(
    c_text,
    "\nСимволи з однаковою ймовірністю"
)

```

Результат:

```

PS D:\Projects\crypto> & C:\msys64\ucrt64\bin\python.exe d:/Projects/crypto/part3.py
Природний текст
H1 = 4.375971 bit

Повторення одного символу
H1 = 0.000000 bit

Символи з однаковою ймовірністю
H1 = 5.044394 bit
PS D:\Projects\crypto> 

```

Проаналізувавши результат то можна побачити що прогноз повністю підтвердився

4. Дослідження залежностей між символами

Наш прогноз:

- H_1 у Γ і Δ не повинні сильно відрізнятися(H_1 залежить тільки від частот окремих символів. А в обох послідовностях частоти однакові)
- H_2 повинні відрізнятися(Тому що тепер враховується не тільки частота а і б, а як вони розташовані один відносно одного)
- Менше H_2 очікуємо у Δ (тому що в Δ наступний символ фактично легко передбачити, А в Γ порядок випадковий, тому наступний символ передбачити складніше)
- Перекривні та неперекривні H_2 можуть відрізнятися(для неперекривних біграм у Δ фактично залишається одна біграма — аб, тому їхня ентропія буде дуже

Перший(символи перемішані у випадковому порядку)

Другий(символи у певному порядку)

```

37 def calculate_h2(text, step):
38
39     frequencies = bigram_frequencies(text, step)
40     total = sum(frequencies.values())
41     h2 = 0
42
43     for count in frequencies.values():
44
45         p = count / total
46         h2 += p * math.log2(p)
47
48     return h2 / 2
49
50
51 def print_statistics(text, name):
52
53     print(name)
54
55     h1 = calculate_h1(text)
56     print(f"H1 = {h1:.6f} bit")
57
58     # Біграми, що перетинаються
59
60     h2_overlap = calculate_h2(text, 1)
61     print(f"H2 (біграми, що перетинаються) = {h2_overlap:.6f} bit")
62
63     # Біграми, що не перетинаються
64
65     h2_non_overlap = calculate_h2(text, 2)
66     print(f"H2 (біграми, що не перетинаються) = {h2_non_overlap:.6f} bit")
67

```

```

69 with open(random, "r", encoding="utf-8") as file:
70
71     random_text = file.read()
72
73
74 with open(order, "r", encoding="utf-8") as file:
75
76     order_text = file.read()
77
78
79 print_statistics(
80     random_text,
81     "\nRandom"
82 )
83
84
85 print_statistics(
86     order_text,
87     "\nOrder"
88 )

```

Результат:

```

PS D:\Projects\crypto> & C:\msys64\ucrt64\bin\python.exe d:/Projects/crypto/part4.py

Random
H1 = 0.998170 bit

H2 (біграми, що перетинаються) = 0.892860 bit
H2 (біграми, що не перетинаються) = 0.864794 bit

Order
H1 = 1.000000 bit

H2 (біграми, що перетинаються) = 0.499988 bit
H2 (біграми, що не перетинаються) = 0.000000 bit
PS D:\Projects\crypto> 

```

Пояснення

У результаті експерименту для послідовності Random отримано $H1 = 0.998170$ біт, що майже дорівнює 1 біту, оскільки символи а і б зустрічаються приблизно з однаковими частотами, але розташовані у випадковому порядку. Через відсутність чіткої закономірності значення $H2$ також залишається високим: 0.892860 для перекривних та 0.864794 для неперекривних біграм.

Для послідовності Order, де символи розташовані періодично (абабаб...), $H1 = 1.000000$ біт, оскільки частоти а і б однакові, але $H2$ значно менша - 0.499988 для перекривних біграм, оскільки після а завжди йде б і навпаки. Для неперекривних біграм отримано $H2 = 0$, оскільки в такій послідовності всі неперекривні пари однакові, тому невизначеність повністю відсутня.

5. Експериментальна оцінка ентропії джерела

Оцінюємо значення для $H^{(10)}$

Лабораторная работа №1

Произвольная часть текста:
конов_и_среди_них_имеется_только_один_который_он_свободен_нарушить_будучи_ф

Использованные буквы:
п.

Порядок n-граммы:
5 символов
15 символов
20 символов
25 символов
30 символов
35 символов
40 символов
45 символов
50 символов

Введенный символ: р

Символ по счету: 2

Номер эксперимента: 50

Неравенство для энтропии:
 $2,09575160417049 < H < 2,67494675029724$

Двоичная таблица угаданных символов:

00000000010000000000000000000000	
00000000010000000000000000000000	
10000000000000000000000000000000	
10000000000000000000000000000000	
00100000000000000000000000000000	

Поле ввода символов:
р

Продолжить Другой

Вероятности:

$q[1] = 0,48$
$q[2] = 0,14$
$q[3] = 0,08$
$q[4] = 0$
$q[5] = 0$
$q[6] = 0$
$q[7] = 0$
$q[8] = 0,06$
$q[9] = 0$
$q[10] = 0,04$
$q[11] = 0,02$
$q[12] = 0,02$
$q[13] = 0$
$q[14] = 0$
$q[15] = 0$
$q[16] = 0,02$
$q[17] = 0$
$q[18] = 0$
$q[19] = 0$
$q[20] = 0,02$
$q[21] = 0$
$q[22] = 0,04$
$q[23] = 0$
$q[24] = 0$
$q[25] = 0$
$q[26] = 0,02$
$q[27] = 0,04$
$q[28] = 0,02$
$q[29] = 0$
$q[30] = 0$
$q[31] = 0$
$q[32] = 0$

Строка состояния:
Вы угадали. Для продолжения опыта нажмите "Продолжить", или "Другой" для выбора другого порядка

Оцінюємо значення для $H^{(20)}$

Лабораторная работа №1

Произвольная часть текста:
ивались_совершенно_

Использованные буквы:

Порядок n-граммы:
5 символов
10 символов
15 символов
20 символов
25 символов
30 символов
35 символов
40 символов
45 символов
50 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 51

Неравенство для энтропии:
 $1,59291637852389 < H < 2,25893665306873$

Двоичная таблица угаданных символов:

10000000000000000000000000000000	
10000000000000000000000000000000	
00001000000000000000000000000000	
00100000000000000000000000000000	
10000000000000000000000000000000	

Поле ввода символов:

Продолжить Другой

Вероятности:

$q[1] = 0,58$
$q[2] = 0,1$
$q[3] = 0,08$
$q[4] = 0$
$q[5] = 0,06$
$q[6] = 0$
$q[7] = 0,04$
$q[8] = 0,02$
$q[9] = 0$
$q[10] = 0$
$q[11] = 0$
$q[12] = 0$
$q[13] = 0,02$
$q[14] = 0$
$q[15] = 0,02$
$q[16] = 0$
$q[17] = 0$
$q[18] = 0$
$q[19] = 0$
$q[20] = 0$
$q[21] = 0$
$q[22] = 0$
$q[23] = 0$
$q[24] = 0$
$q[25] = 0,02$
$q[26] = 0$
$q[27] = 0,02$
$q[28] = 0$
$q[29] = 0,04$
$q[30] = 0$
$q[31] = 0$
$q[32] = 0$

Строка состояния:

[illegible]

Оцінка	Отримане значення
H1	4.375971
H2	3.995763
H ⁽¹⁰⁾	2.09575160417049 < H < 2.67494675029724
H ⁽²⁰⁾	1.59291637852389 < H < 2.25893665306873
H ⁽³⁰⁾	1.824306... < H < 2.576...

1. Чому оцінки ентропії не повинні обов'язково збігатися?

Отримані значення не повинні збігатися, оскільки вони визначаються різними методами та враховують різний обсяг інформації про текст. $N1$ обчислюється за частотами окремих символів, $N2$ - за частотами біграм, а $N(10), N(20), N(30)$

отримуються експериментально для довшого контексту. Крім того, для CoolPinkProgram значення отримуються на основі обмеженої кількості експериментів, тому вони є оцінками і можуть відрізнятися від теоретично розрахованих H_1 та H_2 .

2. Які статистичні залежності враховуються при визначенні H_1 та H_2 ?

При визначенні H_1 враховуються лише частоти появи окремих символів. Тобто враховується, наскільки часто зустрічаються а, о, space та інші символи, але їхній порядок не враховується.

H_2 враховує вже частоти пар сусідніх символів - біграм. Тому ця модель враховує залежність між одним символом та наступним. Наприклад, у природній мові деякі поєднання символів зустрічаються значно частіше за інші.

3. Чому врахування довших залежностей може змінювати оцінку ентропії природної мови?

У природній мові символи не є повністю незалежними. Вибір наступного символу залежить не лише від одного попереднього символу, а часто від декількох попередніх. Наприклад, певна послідовність букв значно підвищує ймовірність появи конкретного наступного символу. Тому зі збільшенням довжини контексту враховується більше закономірностей мови, що дозволяє точніше оцінити її передбачуваність.

4. Яку тенденцію можна спостерігати при переході від H_1 до оцінок із довшим контекстом?

Загалом при переході від H_1 до моделей, які враховують довший контекст, оцінка ентропії зменшується, оскільки враховується більше статистичних залежностей і закономірностей природної мови. При цьому $H(10), H(20), H(30)$ отримані експериментально, тому вони не утворюють строго монотонного ряду: їхні інтервали частково перекриваються.

6. Оцінювання надлишковості джерела

Використаємо формулу надлишковості:

$$R = 1 - \frac{H}{H_{\max}}$$

де $H_{\max} = \log_2 34 = 5.087463$ (33 літери+пробіл)

Тоді надлишковість:

$R_1 = 0.139852 \approx 13.99\%$

$R_2 = 0.214586 \approx 21.46\%$

$R_{10} = 47.42\text{--}58.81\%$

$R_{20} = 55.60\text{--}68.69\%$

$R_{30} = 49.35\text{--}64.14\%$

1. як змінюється оцінка надлишковості при врахуванні залежностей між символами

При врахуванні залежностей між символами оцінка надлишковості зростає. Для H_1 вона становить приблизно **13.99%**, а для H_2 — вже **21.46%**. Це означає, що модель біграм виявляє додаткові закономірності природної мови, які не враховує модель незалежних символів.

2. чому природна мова має ненульову надлишковість

Природна мова має ненульову надлишковість, тому що її символи використовуються нерівномірно, а їхнє розташування підпорядковується мовним закономірностям. Наприклад, одні букви зустрічаються значно частіше за інші, а певні поєднання букв є значно ймовірнішими за інші.

3. який зв'язок існує між передбачуваністю наступних символів тексту, ентропією та надлишковістю джерела

Між передбачуваністю, ентропією та надлишковістю існує прямий зв'язок: чим більш передбачуваним є наступний символ, тим менша ентропія джерела і тим більша його надлишковість. Навпаки, для абсолютно рівномірного та незалежного джерела ентропія була б максимальною, а надлишковість — нульовою.

Аналіз результатів

За результатами виконаних експериментів дайте обґрунтовані відповіді на такі питання:

1. Чому природний текст має меншу ентропію на символ, ніж випадкова рівноймовірна послідовність символів того самого алфавіту?

Природний текст має меншу ентропію, оскільки символи в ньому мають нерівномірний розподіл і повторюються з різною частотою. Наприклад, у нашому `normalized.txt` найчастішим символом виявився пробіл із ймовірністю 0.162167, а частоти інших символів також суттєво відрізняються. У випадковій рівноймовірній послідовності всі символи мають приблизно однакові ймовірності, тому невизначеність є більшою. Це було продемонстровано в експерименті А–Б–В: для повторення одного символу ентропія повинна бути близькою до нуля, для природного тексту — вищою, а для рівноймовірної випадкової послідовності — найбільшою.

2. Яку інформацію про джерело враховує H_2 , але не враховує H_1 ?

H_1 враховує тільки частоти окремих символів, тоді як H_2 враховує ще й залежності між сусідніми символами через частоти біграм. Це добре видно з наших результатів: для `normalized.txt` отримано $H_1 = 4.375971$ біт, а $H_2 = 3.995763$ біт. Зменшення ентропії при переході до біграмної моделі показує, що порядок сусідніх символів містить додаткову інформацію про джерело.

3. Чи можуть два тексти мати однакове H_1 , але різне H_2 ? Наведіть приклад із виконаних експериментів.

Так. Два тексти можуть мати однакові або дуже близькі частоти окремих символів, а отже й близькі H_1 , але різний порядок цих символів. Саме це ми перевіряли в експерименті з файлами `random` та `order`: обидві послідовності побудовані з а і б з однаковими частотами, але в `random` символи розташовані випадково, а в `order` утворюють періодичну послідовність абабаб.... Тому H_1 для них має бути однаковим або дуже близьким, тоді як H_2 відрізняється. Отже, однаковий розподіл окремих символів не означає однакову структуру їхнього

розташування.

4. Як вилучення пробілів вплинуло на отримані значення ентропії? Поясніть отриманий результат.

Вилучення пробілів збільшило H_2 . Для тексту з пробілами ми отримали $H_2 = 3.995763$ біт, а для тексту без пробілів — $H_2 = 4.167906$ біт. Це пояснюється тим, що пробіл є дуже частим символом у природному тексті та утворює характерні біграми, наприклад `i[space]`, `o[space]`, `a[space]`. Після вилучення пробілів частина таких закономірностей зникає, а послідовність символів стає статистично менш структурованою, тому ентропія зростає.

5. Чому оцінки H_2 , отримані за біграмами, що перетинаються та не перетинаються, можуть дещо відрізнятися?

Перекривні біграми формуються зі зміщенням на один символ, тому для тексту довжини NN використовується приблизно $N-1$ біграм. Неперекривні формуються зі зміщенням на два символи, тому їх приблизно $N/2$. Через різний склад вибірки та кількість статистичних даних розподіли біграм можуть трохи відрізнятися. У нашому експерименті для `normalized.txt` отримано 3.995763 біт для перекривних та 3.995578 біт для неперекривних біграм. Різниця становить лише 0.000185 біта, тобто результати практично збігаються.

6. Чому значення H_1 , H_2 , $H^{(10)}$, $H^{(20)}$ та $H^{(30)}$ відрізняються?

Ці показники отримуються для моделей із різною довжиною врахованого контексту. H_1 враховує тільки окремі символи, H_2 — сусідні пари, а $H^{(10)}$, $H^{(20)}$, $H^{(30)}$ отримані експериментально з урахуванням довшого контексту.

У нашому дослідженні:

$H_1 = 4.375971 = 4.375971$ біт;

$H_2 = 3.995763 = 3.995763$ біт;

$H^{(10)}: 2.09575 < H < 2.67495; 2.09575 < H < 2.67495;$

$H^{(20)}: 1.59292 < H < 2.25894; 1.59292 < H < 2.25894;$

$H^{(30)}: 1.82431 < H < 2.57664; 1.82431 < H < 2.57664.$

Таким чином, різні моделі враховують різний обсяг статистичної інформації. Значення $H(10), H(20), H(30)$ також є експериментальними інтервальними оцінками, тому вони не зобов'язані точно збігатися між собою або з H_1 та H_2 .

7. Який із проведених експериментів найкраще демонструє наявність залежностей між послідовними символами?

Найкраще це демонструє експеримент із **random та order**. В обох послідовностях використовуються ті самі символи а і б з однаковими частотами, але їх порядок різний. У **random** порядок випадковий, а в **order** символи строго чергуються: абабаб.... Тому H_1 для них має бути однаковим або близьким, оскільки частоти символів однакові, а H_2 відрізняється через різну структуру сусідніх символів. Саме це показує, що біграмна модель здатна виявляти залежності, яких не бачить H_1 .

8. Як отримані результати пов'язані з можливістю стиснення природного тексту без втрати інформації?

Наші результати показують, що природний текст має нерівномірний розподіл символів та статистичні залежності, тобто він є надлишковим. Для **normalized.txt** $H_1 = 4.375971 = 4.375971$ біт/символ, що менше за максимальне значення для алфавіту з 34 символів:

$$H_{\max} = \log_2 34 \approx 5.0875$$

Ця різниця означає наявність статистичної надлишковості. Саме таку надлишковість можна використовувати для стиснення без втрати інформації: більш частим і передбачуваним символам або послідовностям можна призначати коротші коди, а рідшим — довші. Тому чим краще модель враховує закономірності природної мови, тим більше потенційної надлишковості вона може виявити та використати для стиснення.

Висновки:

У ході лабораторної роботи було досліджено поняття ентропії та надлишковості джерела на прикладі російськомовного тексту. Було встановлено, що ентропія залежить не лише від частот окремих символів, а й від порядку їх розташування та залежностей між ними. Порівняння різних типів послідовностей показало, що рівномірний випадковий розподіл має більшу невизначеність, тоді як повторювані та закономірні послідовності є більш передбачуваними. Використання біграм дозволило врахувати залежності між сусідніми символами, а збільшення довжини контексту - виявити додаткові закономірності природної мови. Також було встановлено, що природний текст має статистичну надлишковість, яка виникає через нерівномірний розподіл символів та закономірності їх поєднання. Отримані результати показали важливість урахування структури та передбачуваності тексту під час оцінювання його ентропії, а також продемонстрували, що виявлена надлишковість може бути використана для стиснення текстової інформації без втрати даних.